



CICLO FORMATIVO DE GRADO SUPERIOR EN

.....

PROYECTO FIN DE CICLO

Título: Nómada

Alumno: Pablo Santamaría Gonzalez

Nº Alumno: 355741

DECLARACIÓN JURADA DE AUTORÍA

D. Pablo Santamaria Gonzalez, con DNI 51785597Q como autor de este documento académico, titulado:
Nómada
y presentado como Proyecto Fin de Ciclo para la obtención del Ciclo Formativo de Grado Superior en Desarrollo de aplicaciones multiplataforma,

DECLARO QUE

Soy el único autor del trabajo, con la excepción de referencias a contenidos o ideas de otros autores, en cuyo caso han sido explícitamente citados.

El trabajo remitido es un documento original, que no ha sido publicado, ya sea total o parcialmente, ni presentado para obtención de un título académico en ninguna institución académica u organización.

No he trasgredido ninguna norma universitaria con respecto al plagio ni a las leyes establecidas que protegen la propiedad intelectual.

Soy consciente de que el hecho de no respetar estos extremos es una falta grave de integridad académica y podrá ser objeto de sanciones.

En Madrid, a ____ de _____ de 20____

(firma)

(firma)

(firma)

Fdo.:

Fdo.:

Fdo.:

RESUMEN

La aplicación **Nómada** ha sido diseñada como una plataforma móvil orientada a la búsqueda y recopilación de recursos de aprendizaje sobre cualquier temática de interés. Su objetivo principal es centralizar el acceso a contenido educativo procedente de fuentes externas fiables, como vídeos, podcasts, libros o artículos, obtenidos mediante el uso de distintas APIs públicas, y combinarlo con aportaciones realizadas por la propia comunidad de usuarios. La aplicación permitirá a los usuarios realizar búsquedas por palabras clave y visualizar los resultados organizados por tipo de recurso, diferenciando entre contenido oficial y contenido generado por los usuarios. Además, los usuarios registrados podrán añadir sus propios recursos, gestionar una lista de favoritos y consultar su historial de búsquedas. Nómada contará con una estructura basada en varias pantallas que facilitarán la navegación, la consulta de información y la gestión del perfil de usuario, ofreciendo una experiencia clara, organizada y orientada al aprendizaje autodidacta.

ABSTRACT

The **Nómada** application has been designed as a mobile platform focused on searching for and collecting learning resources on any topic of interest. Its main objective is to centralize access to educational content from reliable external sources, such as videos, podcasts, books, or articles, obtained through the use of various public APIs, and to combine this content with contributions made by the user community. The application allows users to perform keyword-based searches and view results organized by resource type, clearly differentiating between official content and user-generated content. In addition, registered users can add their own resources, manage a list of favorites, and consult their search history. Nómada is structured around multiple screens that facilitate navigation, information consultation, and user profile management, offering a clear, organized, and self-directed learning-oriented user experience.

Contenido

1.	TÍTULO PRINCIPAL (ejemplo)	¡Error! Marcador no definido.
1.1.	Título del apartado (ejemplo)	¡Error! Marcador no definido.
2.	INTRODUCCIÓN	6
3.	REQUISITOS FUNCIONALES.....	7
4.	DISEÑO DE LA SOLUCIÓN.....	9
4.1.	Mapa de navegación.....	9
4.2.	Modelo de base de datos.....	10
4.3.	Diseño de la arquitectura	10
4.4.	Lenguajes utilizados.....	13
5.	DISEÑO TÉCNICO.....	14
5.1.	Libro de estilo	14
5.2.	Definición de componentes	17
5.3.	Diseño de tablas de la base de datos.....	19
6.	DEMOSTRACIÓN	20
7.	BIBLIOGRAFÍA	20
8.	ANEXOS	20
8.1.	Código fuente.....	20

ÍNDICE DE ILUSTRACIONES

Ilustración 1 : Título de la imagen. Fuente: **¡Error! Marcador no definido.**

ÍNDICE DE TABLAS

Tabla 1: Título de la tabla. Fuente: 5

Tabla 1: Título de la tabla. Fuente:

Columna 1	Columna 2	Columna 3

2. INTRODUCCIÓN

En la actualidad, el acceso a la información y al aprendizaje autodidacta se ha convertido en un aspecto fundamental tanto a nivel personal como profesional. Internet ofrece una enorme cantidad de recursos educativos en distintos formatos, como vídeos, podcasts, artículos o libros, distribuidos en múltiples plataformas y servicios. Sin embargo, esta abundancia de información también supone una dificultad a la hora de encontrar contenidos relevantes, fiables y bien organizados sobre un tema concreto. En este contexto, surge la necesidad de herramientas que permitan centralizar y estructurar el acceso al conocimiento, facilitando el aprendizaje continuo y personalizado.

Desde un punto de vista personal, siempre me he considerado una persona curiosa, con interés por aprender sobre temas muy diversos. En muchas ocasiones, al querer profundizar en un área concreta, me he encontrado con la dificultad de tener que consultar diferentes aplicaciones y páginas web para obtener información completa y variada. Esta situación me llevó a plantear la idea de crear una aplicación que me permitiera tener todos esos recursos reunidos en un mismo lugar. Además, considero especialmente importante poder acceder a distintos puntos de vista sobre un mismo tema, no solo a través de fuentes oficiales, sino también mediante las aportaciones de otras personas, ya que esto enriquece el proceso de aprendizaje y ofrece una visión más amplia y realista del conocimiento.

Desde el punto de vista técnico y formativo, la elección de este proyecto está directamente relacionada con los conocimientos adquiridos durante el ciclo formativo de Desarrollo de Aplicaciones Multiplataforma. Este proyecto me permite aplicar de forma práctica conceptos fundamentales como el desarrollo de aplicaciones móviles, la integración de APIs externas, la gestión de bases de datos y la implementación de la lógica de negocio. Asimismo, el desarrollo de una aplicación completa desde cero me brinda la oportunidad de trabajar todas las fases de un proyecto real, desde el diseño inicial hasta la implementación y estructuración final, consolidando así las competencias técnicas adquiridas a lo largo del ciclo.

El objetivo principal de este trabajo es desarrollar una aplicación móvil funcional que permita buscar, organizar y consultar recursos de aprendizaje sobre cualquier temática, combinando información obtenida de fuentes externas fiables con contenido generado por los propios usuarios. Como objetivos secundarios, se pretende implementar un sistema de usuarios con gestión de perfiles, una base de datos estructurada que permita almacenar aportaciones, favoritos e historial de búsquedas, y una interfaz intuitiva que facilite la navegación y el acceso a la información. De esta forma, el proyecto Nómada busca no solo resolver una necesidad personal, sino también demostrar la aplicación práctica de los conocimientos técnicos adquiridos durante la formación, ofreciendo una solución útil, escalable y orientada al aprendizaje continuo.

1. REQUISITOS FUNCIONALES

La aplicación **Nómada** deberá ofrecer un conjunto de funcionalidades orientadas a facilitar el aprendizaje autónomo mediante la búsqueda y consulta de recursos educativos, así como la aportación de información por parte de los propios usuarios. A continuación, se detallan los requisitos funcionales que definen el comportamiento y las acciones que podrá realizar la aplicación desde el punto de vista del usuario.

3.1 Gestión de usuarios

La aplicación requerirá que todos los usuarios se registren para poder acceder a sus funcionalidades.

El registro de usuario solicitará, como mínimo, los siguientes datos:

- Nombre
- Dirección de correo electrónico
- Contraseña

Los usuarios registrados podrán iniciar sesión en la aplicación mediante su correo electrónico y contraseña.

Existirán dos tipos de usuarios:

Usuario estándar, que podrá utilizar las funcionalidades principales de la aplicación.

Administrador, que contará con permisos adicionales de control y moderación de contenido.

Los usuarios podrán acceder a una pantalla de perfil donde visualizarán y podrán modificar sus datos personales.

3.2 Búsqueda de contenido

La aplicación dispondrá de un **buscador principal** que permitirá realizar búsquedas introduciendo palabras clave o temas de interés.

Al realizar una búsqueda, la aplicación mostrará todos los resultados relacionados con el término introducido.

Los resultados de búsqueda se presentarán organizados y diferenciados en dos bloques:

Recursos oficiales, obtenidos a través de APIs externas.

Aportaciones de usuarios, generadas dentro de la propia aplicación.

La aplicación permitirá filtrar o identificar los resultados según el tipo de recurso.

3.3 Visualización de recursos oficiales

Los usuarios podrán consultar recursos oficiales relacionados con un tema determinado.

Los tipos de recursos oficiales que se mostrarán en la aplicación serán:

- Artículos
- Vídeos
- Podcasts
- Libros
- Cursos

Al seleccionar un recurso, el usuario accederá a una pantalla de detalle donde se mostrará la información principal del mismo (título, descripción y tipo de recurso).

3.4 Aportaciones de usuarios

Los usuarios registrados podrán añadir aportaciones propias en forma de texto.

Las aportaciones podrán estar asociadas:

- A un tema concreto.
- A un recurso oficial específico.

No se permitirá la inclusión de enlaces externos en las aportaciones de los usuarios.

Los usuarios podrán:

- Crear nuevas aportaciones.
- Editar sus propias aportaciones.
- Eliminar sus propias aportaciones.

Las aportaciones de los usuarios se mostrarán en un apartado independiente del contenido oficial, claramente diferenciado.

3.5 Favoritos

La aplicación permitirá a los usuarios guardar recursos y aportaciones como **favoritos**.

Los usuarios podrán acceder a una pantalla específica donde se mostrarán todos los elementos marcados como favoritos.

Desde esta pantalla, los usuarios podrán eliminar elementos de su lista de favoritos.

3.6 Historial de uso

La aplicación mantendrá un **historial de búsquedas y contenidos visualizados** por cada usuario.

Los usuarios podrán acceder a una pantalla donde se muestre su historial. El historial podrá ser eliminado manualmente por el usuario, total o parcialmente.

3.7 Reporte y moderación de contenido

Los usuarios podrán **reportar aportaciones** creadas por otros usuarios.

Al reportar un contenido, este quedará marcado para su revisión.

El administrador podrá:

- Revisar los contenidos reportados.
- Decidir si el contenido debe mantenerse o eliminarse.

Esta funcionalidad permitirá garantizar un uso adecuado de la aplicación y la calidad de la información compartida.

3.8 Pantallas de la aplicación

La aplicación contará con las siguientes pantallas:

- Pantalla de inicio
- Pantalla de registro e inicio de sesión
- Pantalla de búsqueda
- Pantalla de resultados de búsqueda
- Pantalla de detalle de recurso
- Pantalla de aportaciones de usuarios

- Pantalla de perfil de usuario
- Pantalla de favoritos
- Pantalla de historial
- Pantalla de ajustes

3.9 Funcionalidades del administrador

El administrador podrá gestionar las aportaciones de los usuarios.

Podrá revisar y eliminar contenidos reportados.

Tendrá acceso a las funcionalidades necesarias para garantizar el correcto funcionamiento de la aplicación.

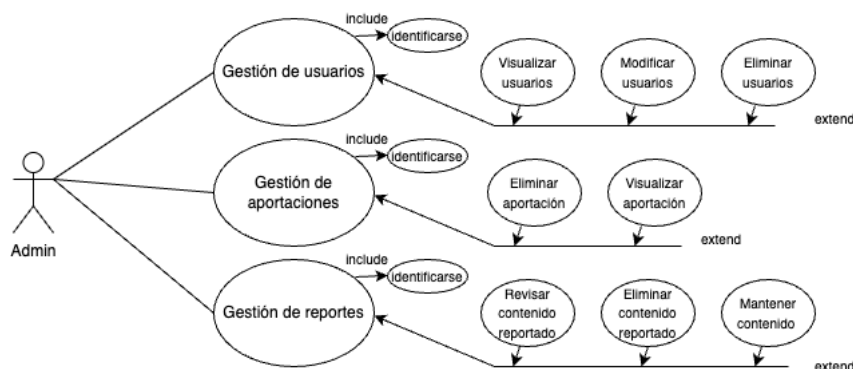
2. DISEÑO DE LA SOLUCIÓN

Su diseño el lógico o conceptual, sin ser necesario detallar técnicamente como se desarrolla la solución, donde se realizará en el apartado Diseño técnico de la solución.

2.1. Mapa de navegación

Basado en los requisitos funcionales, en esta sección se especifican el diagrama de casos de uso de las pantallas y las interacciones que tendrá la aplicación para asegurar que dará respuesta a las funcionalidades planteadas.

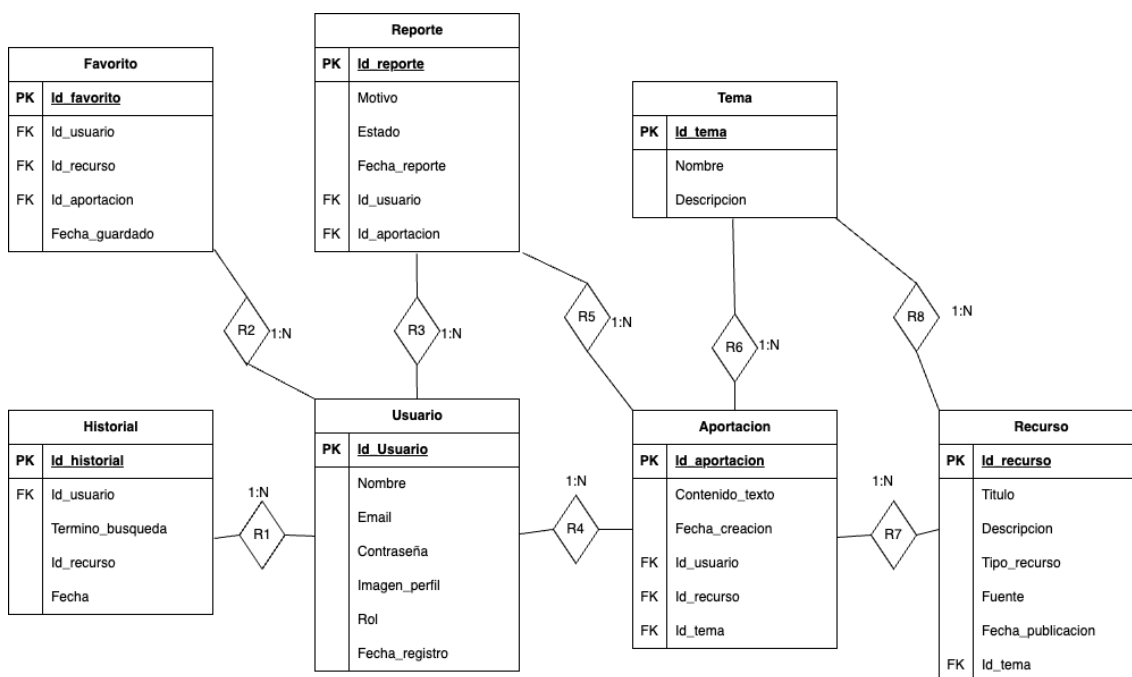




2.2. Modelo de base de datos

Para el diseño de Modelo de base de datos se ha utilizado la plataforma draw.io, identificando las entidades necesarias, los atributos más importantes, las cardinalidades y las relaciones entre las entidades.

Diseño Entidad / Relación:



Entidades:

- Usuario:**
 Registro de los usuarios de la aplicación. Esta entidad almacena la información básica necesaria para la identificación y gestión de los usuarios, como el nombre, correo electrónico, contraseña, imagen de perfil, rol dentro del sistema (usuario o administrador) y la fecha de registro. Se utiliza para controlar el acceso a la aplicación y asociar las acciones realizadas (aportaciones, favoritos, historial, reportes).
- Tema:**
 Registro de los distintos temas o categorías sobre los que se organiza el contenido de la aplicación. Cada tema dispone de un nombre y una descripción que permiten clasificar tanto los recursos como las aportaciones de los usuarios, facilitando la búsqueda y navegación por la información.

- **Recurso:**
Registro de los recursos informativos disponibles en la aplicación. Un recurso contiene datos como el título, descripción, tipo de recurso, fuente, fecha de publicación y el tema al que pertenece. Esta entidad se utiliza para ofrecer información estructurada y oficial sobre los distintos temas disponibles.
- **Aportación:**
Registro de las aportaciones realizadas por los usuarios. Cada aportación incluye el contenido en formato texto, la fecha de creación y la relación con el usuario que la crea, el tema al que pertenece y, en su caso, el recurso asociado. Permite a los usuarios compartir opiniones y puntos de vista sobre los temas tratados.
- **Favorito:**
Registro de los recursos o aportaciones que un usuario ha marcado como favoritos. Esta entidad permite guardar información de interés para su posterior consulta, asociando el usuario con el recurso o aportación correspondiente y la fecha en la que se realizó la acción.
- **Historial:**
Registro del historial de búsquedas y consultas realizadas por el usuario dentro de la aplicación. Almacena información como el término de búsqueda, el recurso consultado, la fecha y el usuario asociado. Se utiliza para mejorar la experiencia del usuario y permitir la revisión de búsquedas anteriores.
- **Reporte:**
Registro de los reportes realizados por los usuarios sobre aportaciones que consideran inapropiadas. Contiene información como el motivo del reporte, el estado del mismo, la fecha en la que se realiza, el usuario que reporta y la aportación reportada. Esta entidad es clave para la moderación de contenido por parte de los administradores.

2.3. Diseño de la arquitectura

La arquitectura de la aplicación se ha diseñado siguiendo un modelo **cliente-servidor**, con el objetivo de separar de forma clara las responsabilidades entre la aplicación móvil, la lógica de negocio y la persistencia de los datos. Este enfoque permite una mayor escalabilidad, mantenibilidad y seguridad del sistema, además de facilitar el desarrollo y la comprensión de la solución.

De forma general, la arquitectura de la aplicación está compuesta por los siguientes componentes principales:

a) BACK-END

El back-end de la aplicación se implementará mediante una **API REST desarrollada en Java con Spring Boot**, encargada de gestionar toda la lógica de negocio del sistema. Este componente será el único con acceso directo a la base de datos y actuará como intermediario entre la aplicación móvil y los datos almacenados.

Entre las principales funcionalidades del back-end se encuentran:

- Registro e inicio de sesión de usuarios.
- Gestión de usuarios y control de roles (usuario y administrador).
- Gestión de temas y recursos asociados.
- Gestión de aportaciones realizadas por los usuarios.
- Gestión de favoritos e historial de búsquedas.
- Gestión de reportes y moderación de contenido.

- Consumo de APIs externas para la obtención de recursos oficiales.

El back-end será responsable de procesar las peticiones recibidas, validar los datos, aplicar las reglas de negocio y devolver las respuestas en formato **JSON** a la aplicación móvil.

b) FRONT-END

El front-end de la aplicación está compuesto por una **aplicación móvil Android desarrollada en Kotlin**, utilizando archivos XML para la definición de la interfaz de usuario. La aplicación proporcionará una experiencia de usuario intuitiva y clara, permitiendo al usuario interactuar con las distintas funcionalidades del sistema.

Desde la aplicación móvil, el usuario podrá:

- Registrarse e iniciar sesión.
- Realizar búsquedas por palabra o por tema.
- Consultar recursos oficiales y aportaciones de otros usuarios.
- Añadir y gestionar sus propias aportaciones.
- Guardar elementos en favoritos.
- Consultar su historial de búsquedas.
- Reportar contenido inapropiado.

La comunicación entre el front-end y el back-end se realizará mediante peticiones HTTP a la API REST, intercambiando la información en formato JSON.

c) SERVIDOR DE APLICACIONES

El servidor de aplicaciones está basado en **Spring Boot**, el cual incluye un **servidor Tomcat embebido**. Este servidor es el encargado de ejecutar la API REST y gestionar las peticiones HTTP provenientes de la aplicación móvil.

El servidor de aplicaciones se ocupa de:

- Recibir las solicitudes de la aplicación móvil.
- Redirigir las peticiones a los controladores correspondientes.
- Ejecutar la lógica de negocio definida.
- Generar y devolver las respuestas al cliente.

Al tratarse de un entorno académico, el servidor se ejecutará de forma local durante el desarrollo y las pruebas de la aplicación.

d) SERVIDOR WEB

En esta arquitectura, el servidor web se encuentra **integrado dentro del propio servidor de aplicaciones**, gracias al uso de Spring Boot y su servidor Tomcat embebido. No se utiliza un servidor web independiente, ya que la aplicación no dispone de una interfaz web tradicional, sino que se comunica exclusivamente a través de la API REST consumida por la aplicación móvil.

Este servidor web integrado se encarga de gestionar las peticiones HTTP y facilitar la comunicación entre el front-end y el back-end de manera eficiente y segura.

e) GESTOR DE BASE DE DATOS

El gestor de base de datos seleccionado para la aplicación es **MariaDB**, el cual se utilizará para almacenar toda la información necesaria para el correcto funcionamiento del sistema. La base de datos contendrá las tablas relacionadas con usuarios, temas, recursos, aportaciones, favoritos, historial de búsquedas y reportes.

La conexión a la base de datos se realizará **exclusivamente desde el back-end**, garantizando así una mayor seguridad y control sobre el acceso a los datos. El uso de MariaDB permite realizar operaciones de lectura y escritura de forma eficiente, cumpliendo con los requisitos funcionales definidos.

f) SEGURIDAD

La arquitectura contempla diversas medidas de seguridad para proteger la información y garantizar la integridad del sistema:

- Autenticación de usuarios mediante credenciales.
- Control de acceso basado en roles (usuario y administrador).
- Validación y sanitización de los datos recibidos desde el front-end.
- Comunicación segura entre la aplicación móvil y el back-end mediante peticiones controladas a la API REST.

Finalmente, para facilitar la comprensión de la arquitectura del sistema, se ha incluido una **infografía de la arquitectura general**, donde se representan todos los componentes descritos y las relaciones entre ellos, mostrando el flujo de comunicación entre la aplicación móvil, la API REST, la base de datos y las APIs externas utilizadas.

2.4. Lenguajes utilizados

Para el desarrollo de la solución **Nómada**, se ha seleccionado un conjunto de lenguajes y formatos de datos que permiten un desarrollo multiplataforma eficiente. A continuación, se detallan los lenguajes utilizados y el objetivo de cada uno dentro del sistema:

a) Java (Back-end)

Es el lenguaje principal utilizado para el desarrollo de la lógica de negocio en el servidor. Gracias a su robustez y su arquitectura orientada a objetos, **Java** es la base del framework **Spring Boot**.

Objetivo: Implementar la API REST que gestiona el control de roles (usuario/administrador), la lógica de moderación de reportes y la conexión segura con el gestor de base de datos. Es el motor que procesa las reglas de negocio antes de enviar la información al cliente.

b) Kotlin (Front-end)

Es un lenguaje de programación moderno, conciso y totalmente interoperable con Java, designado por Google como el lenguaje preferido para el desarrollo en Android.

Objetivo: Desarrollar la aplicación móvil nativa. **Kotlin** se encarga de gestionar la lógica de la interfaz, procesar las búsquedas por palabra clave, gestionar el almacenamiento local de favoritos y realizar las llamadas asíncronas a la API REST para obtener los recursos educativos.

c) SQL (Gestión de Datos)

Aunque el sistema utiliza **MariaDB** como motor, el lenguaje de consulta estructurado (**SQL**) es la herramienta utilizada para la comunicación con la base de datos.

Objetivo: Definir y manipular la persistencia de la información. Se utiliza para realizar operaciones CRUD (Crear, Leer, Actualizar y Borrar) sobre las siete entidades principales: Usuarios, Temas, Recursos, Aportaciones, Favoritos, Historial y Reportes.

d) JSON (Intercambio de Información)

JSON (JavaScript Object Notation) es un formato ligero de intercambio de datos, fácil de leer para los humanos y sencillo de procesar para las máquinas.

Objetivo: Actuar como el "idioma común" entre el Front-end y el Back-end. Toda la información de los recursos oficiales obtenidos de APIs externas y las aportaciones de los usuarios se encapsulan en objetos JSON para ser transferidos de forma eficiente mediante peticiones HTTP.

3. DISEÑO TÉCNICO

Los siguientes apartados describen con más detalle y con descripción técnica como se ha construido la solución.

3.1. Libro de estilo

El libro de estilo tiene como finalidad garantizar la coherencia gráfica del sistema, asegurando una identidad visual homogénea y una experiencia de usuario clara, intuitiva y profesional.

La aplicación ha sido diseñada siguiendo una línea estética minimalista, limpia y moderna. Se ha priorizado la claridad en la presentación de la información, la correcta jerarquización de los contenidos y la facilidad de navegación, evitando la sobrecarga visual y el uso innecesario de elementos decorativos.

Rejilla y estructura de diseño

La estructura de la interfaz se ha desarrollado utilizando **ConstraintLayout** como contenedor principal en las distintas pantallas de la aplicación. Este sistema permite organizar los elementos de forma flexible, garantizando la correcta adaptación a distintos tamaños y resoluciones de dispositivos Android.

Se han aplicado márgenes laterales estándar de 16dp y un espaciado vertical uniforme entre componentes, lo que proporciona equilibrio visual y consistencia en toda la aplicación. Para las dimensiones estructurales se utilizan unidades dp, mientras que para la tipografía se emplean unidades sp, asegurando así una correcta escalabilidad según la configuración de accesibilidad del usuario.

Este enfoque garantiza una experiencia coherente y adaptable, independientemente del dispositivo desde el que se acceda a la aplicación.

Logotipo

El logotipo representa la identidad visual del proyecto y actúa como elemento identificativo principal. Se emplea en la pantalla de inicio (Splash Screen), en las pantallas de autenticación y, cuando resulta adecuado, en la barra superior de navegación.

Se han establecido normas claras para su utilización: se deben respetar siempre sus proporciones originales, no modificar los colores corporativos definidos en la paleta oficial y evitar cualquier tipo de deformación o efecto que altere su diseño. Asimismo, se mantiene un margen de seguridad alrededor del logotipo para asegurar su correcta visualización.

Tipografía

La tipografía seleccionada para la aplicación es Calibri, fuente predeterminada en Android Studio y ampliamente optimizada para dispositivos móviles. Esta tipografía ofrece alta legibilidad, buena adaptación a distintas densidades de pantalla y compatibilidad multilinguaje.

Se ha definido una jerarquía tipográfica clara con el fin de diferenciar visualmente los distintos niveles de información:

Elemento	Tamaño
Títulos principales	28sp – 30sp
Subtítulos / Categorías	18sp – 20sp
Texto descriptivo	14sp – 16sp
Texto secundario	12sp
Botones	16sp – 18sp

La utilización de la unidad sp permite que el texto se adapte a las preferencias de tamaño configuradas por el usuario, mejorando la accesibilidad del sistema.

Paleta de colores

La identidad cromática de la aplicación se basa en una combinación de tonos azules profundos, manteniendo una estética elegante y profesional.

Los colores definidos son los siguientes:

Nombre	Código HEX	Función principal
Midnight Blue	#16253D	Color primario
Dusk	#002C54	Color secundario
Golden	#EFB509	Color de acento
Bronze	#CD7213	Complementario
Blanco	#FFFFFF	Fondo principal
Negro	#000000	Texto principal

Imágenes y gráficos

Las imágenes utilizadas en la aplicación han sido seleccionadas y optimizadas para su correcta visualización en dispositivos móviles. Se evita el uso de imágenes de baja resolución o deformadas, priorizando recursos visuales claros y bien iluminados.

Para mantener coherencia estética, las imágenes incorporan bordes ligeramente redondeados (entre un 8% y un 10%), alineándose con el estilo general de la aplicación.

Componentes gráficos y funcionales

Los elementos interactivos de la aplicación mantienen una línea coherente con la identidad visual definida. Los botones presentan bordes ligeramente redondeados (6dp) y utilizan el color secundario (#002C54) como fondo principal, reservando el color Golden (#EFB509) para acciones destacadas. El texto de los botones se muestra en color blanco (#FFFFFF) para asegurar contraste.

Los campos de entrada de texto se implementan mediante componentes `TextInputLayout`, permitiendo validación inmediata y mostrando mensajes de error claros cuando es necesario. En situaciones que requieren confirmación del usuario, como la eliminación de datos, se emplean diálogos de advertencia que bloquean la acción hasta recibir confirmación explícita.

Para la visualización de información estructurada se utiliza `RecyclerView` junto con `CardView`, permitiendo una presentación organizada, con separación uniforme entre elementos y ligera elevación para aportar profundidad visual.

5.2. Definición de componentes

En este apartado se describen los principales componentes técnicos que forman parte del proyecto, tanto en el lado del Front-end como en el Back-end. Estos componentes constituyen las piezas fundamentales que permiten el funcionamiento completo del sistema, garantizando la comunicación entre la aplicación móvil, el servidor y la base de datos.

La arquitectura implementada sigue un modelo cliente-servidor, donde la aplicación móvil Android actúa como cliente y consume los servicios REST desarrollados en el backend mediante peticiones HTTP en formato JSON.

A continuación, se detallan algunos de los componentes funcionales más relevantes del sistema.

Registro de usuario

El proceso de registro comienza en la aplicación móvil, donde el usuario introduce sus datos en la pantalla correspondiente (nombre, correo electrónico y contraseña).

Una vez completado el formulario, la aplicación realiza una petición HTTP de tipo POST a la API REST del servidor, enviando la información en formato JSON al endpoint correspondiente (por ejemplo, /api/usuarios/registro).

En el backend, el controlador recibe la solicitud y delega la operación en la capa de servicio. Esta capa se encarga de validar los datos recibidos, comprobar que el correo electrónico no esté previamente registrado y aplicar las reglas de negocio definidas.

Si la validación es correcta, el servicio utiliza el repositorio correspondiente para realizar la inserción del nuevo usuario en la base de datos PostgreSQL mediante una operación INSERT.

En caso de producirse algún error (por ejemplo, usuario ya existente o fallo en la base de datos), el servidor devuelve una respuesta HTTP con el código de error correspondiente y un mensaje explicativo. La aplicación móvil interpreta dicha respuesta y muestra un mensaje informativo al usuario.

Inicio de sesión

El inicio de sesión se realiza desde la pantalla de autenticación de la aplicación móvil. El usuario introduce su correo electrónico y contraseña, tras lo cual la aplicación envía una petición POST al endpoint de autenticación del backend.

El backend recibe las credenciales, verifica su validez comparándolas con los datos almacenados en la base de datos y, si son correctas, genera la respuesta correspondiente (por ejemplo, confirmación de acceso o token de sesión si se implementa).

En caso de credenciales incorrectas, el servidor devuelve un mensaje de error que es gestionado por la aplicación para informar al usuario.

Consulta de recursos

Cuando el usuario accede al listado de recursos, la aplicación realiza una petición HTTP GET al endpoint correspondiente (por ejemplo, /api/recursos).

El backend procesa la solicitud en el controlador, consulta la base de datos mediante el repositorio y obtiene la información almacenada en la tabla correspondiente.

Los datos recuperados se transforman en formato JSON y se envían como respuesta al cliente. La aplicación móvil recibe la información y la muestra en pantalla utilizando un RecyclerView para presentar los elementos en formato lista.

Este componente permite la visualización dinámica de contenido almacenado en la base de datos.

Creación de aportaciones

El sistema permite a los usuarios añadir nuevas aportaciones. Desde la pantalla correspondiente, el usuario introduce la información requerida y confirma la operación.

La aplicación envía una petición POST al backend con los datos en formato JSON. En el servidor, el controlador recibe la solicitud y la capa de servicio valida la información antes de proceder a su almacenamiento.

Si los datos son correctos, se realiza la inserción en la base de datos PostgreSQL. Tras completarse el proceso correctamente, el servidor devuelve una respuesta de confirmación, permitiendo a la aplicación actualizar el listado de contenido.

Gestión de favoritos

Cuando un usuario marca un recurso como favorito, la aplicación envía una petición POST o PUT al backend indicando el identificador del usuario y del recurso.

El servidor registra la relación en la tabla correspondiente (por ejemplo, tabla intermedia usuario_recurso). Si el usuario decide eliminar el favorito, se realiza una petición DELETE que elimina dicha relación en la base de datos.

Este componente permite mantener una relación personalizada entre el usuario y los contenidos almacenados.

Gestión de errores y validaciones

Todos los componentes descritos incluyen un sistema de validación tanto en el cliente como en el servidor.

En el lado del cliente, se validan los campos obligatorios antes de enviar la solicitud al backend. En el servidor, se vuelven a validar los datos para garantizar la integridad del sistema y evitar inconsistencias en la base de datos.

Las respuestas del servidor incluyen códigos de estado HTTP que permiten a la aplicación interpretar correctamente el resultado de cada operación.

5.3. Diseño de tablas de la base de datos

En base al modelo Entidad/Relación, se especifica el modelo relacional en base a las reglas de transformación que sean de aplicación, y las formas normales si fueran necesarias.

El resultado debe especificar todas las tablas que se vayan a crear en la base de datos, con sus nombres, campos, tipos de datos, descripción, etc.

Se incluye a continuación un ejemplo de uso.

Tabla 2: Título de la tabla. Fuente:

Columna 1	Columna 2	Columna 3

Extensión orientativa 12-15 hojas

6. DEMOSTRACIÓN

Se especifican las instrucciones para instalar, configurar, ejecutar y utilizar la demo en base a los ficheros proporcionados.

Extensión máxima 1 hoja

7. BIBLIOGRAFÍA

Incluir las referencias (bibliográficas, de webs, etc.) que se hayan utilizado

Extensión máxima 1 hoja

8. ANEXOS

8.1. Código fuente

Se deben **cargar en Canvas** todos los ficheros de código utilizados en la web / app (.html, .js, .css, .sql, .java...)

En este apartado únicamente se listan y detallan los ficheros que se entregar en el proyecto, así como una breve descripción de cada fichero.

En ningún caso se debe copiar en este apartado las líneas de código en sí.

Extensión máxima 1 hoja

Fin del documento